

# Day 118: FastAPI Fundamentals

## 1. Day number

Day 118

## 2. Topic name

**REST APIs with FastAPI**

Today you will learn how to create a very small API using Python.

---

## 3. Connection

Earlier, you learned how Python can **consume an external API**:

```
Your Python program
    ↓
sends request
    ↓
External API
    ↓
returns JSON
```

Today you will build the other side.

Instead of being the client, your Python program will act as the **server**:

```
Client
    ↓
request
    ↓
Your FastAPI application
    ↓
Python function
    ↓
JSON response
```

---

## 4. Important topics

### REST API

A **REST API** is a way for programs to communicate over HTTP using URLs and standard request methods.

For example:

```
GET /products
```

could mean:

| Give me a list of products.

And:

GET /products/5

could mean:

| Give me product number 5.

---

## Client

The **client** sends the request.

Examples:

Browser  
Mobile app  
Frontend application  
Python program  
Postman

---

## Server

The **server** receives requests and sends responses.

Today:

FastAPI application

will act as the server.

---

## HTTP

**HTTP** is the protocol commonly used for communication between web clients and servers.

Conceptually:

Client  
↓ HTTP request  
Server  
↓ HTTP response  
Client

## GET

GET is commonly used when the client wants to **retrieve information**.

Example:

```
GET /products
```

Meaning:

Give me product information.

---

## POST

POST is commonly used when the client wants to **send data** to the server, often to create something.

For example:

```
POST /products
```

could send:

```
{  
  "name": "Keyboard",  
  "price": 1500  
}
```

For today's exercise, focus mainly on **GET**.

---

## Request

A **request** is what the client sends to the server.

It may contain:

```
HTTP method  
URL  
parameters  
headers  
body
```

For today's simple GET request, think mainly about:

```
method + URL + parameter
```

---

## Response

A **response** is what the server sends back.

For an API, the response is often JSON.

For example:

```
{  
  "message": "Hello"  
}
```

---

## JSON

JSON is a common format for API responses.

Python:

```
dictionary
```

can often become:

```
JSON object
```

automatically when returned by FastAPI.

---

## FastAPI

**FastAPI** is a Python framework used for building APIs.

Conceptually:

```
Python function  
  +  
FastAPI endpoint  
  ↓  
Web API
```

---

## Endpoint

An **endpoint** is a specific API URL plus an HTTP method.

For example:

```
GET /hello
```

is one endpoint.

Another could be:

```
GET /products/5
```

## 5. Foundational notes

Suppose you normally write a Python function:

```
function:
    receive name
    create message
    return message
```

FastAPI allows that Python function to be connected to a web URL.

Conceptually:

```
/hello
  ↓
Python function
  ↓
return dictionary
  ↓
JSON response
```

So the main idea is not:

FastAPI replaces Python functions.

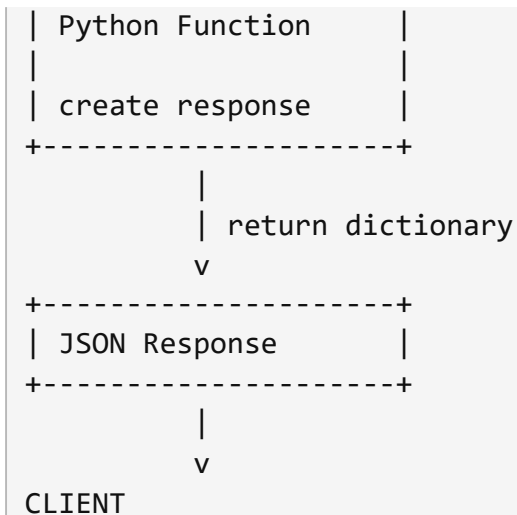
It is:

FastAPI connects Python functions to HTTP requests.

## 6. API request → Python function → response

Here is the basic flow:

```
CLIENT
|
| GET /hello
v
+-----+
| FastAPI Server |
+-----+
      |
      | finds matching endpoint
      v
+-----+
```



For example:

```
GET /hello
```

might trigger a Python function that returns:

```
{
  "message": "Hello"
}
```

---

## 7. Easy FastAPI example

The smallest FastAPI idea looks roughly like:

```
from fastapi import FastAPI

app = FastAPI()
```

Now imagine defining an endpoint:

```
GET /hello
```

and connecting it to a Python function.

Conceptually:

```
GET /hello
  ↓
hello()
  ↓
{"message": "Hello"}
```

The important relationship is:

```
URL
↓
endpoint
↓
Python function
↓
JSON
```

Do not worry about writing the complete working application yet.

---

## 8. Path parameter

A **path parameter** is a value placed directly inside the URL path.

Example:

```
GET /products/5
```

Here:

```
5
```

could be the product ID.

Conceptually:

```
/products/{product_id}
```

means:

Whatever value appears here should be passed to the Python function.

Examples:

```
/products/1
/products/5
/products/100
```

The Python function receives:

```
product_id
```

and can use it to create the response.

---

## 9. Query parameter

A **query parameter** usually appears after ? in a URL.

Example:

```
GET /hello?name=Ravi
```

Here:

```
name=Ravi
```

is a query parameter.

Conceptually:

```
/hello?name=Ravi
      ↓
Python receives:
name = "Ravi"
```

Then the response might be:

```
{
  "message": "Hello Ravi"
}
```

---

## 10. Path parameter vs query parameter

A simple beginner distinction:

```
Path parameter
→ usually identifies a specific resource

Query parameter
→ usually modifies or filters the request
```

Examples:

```
/products/10
```

10 identifies a product.

But:

```
/products?category=books
```

asks:

Give me products filtered by category.

For today's lesson, use either one.

---

## 11. Problem statement

Create a tiny FastAPI application with one GET endpoint.

Choose one of these two versions.

### Option A — Path parameter

Create:

```
GET /students/{student_id}
```

Example request:

```
GET /students/3
```

Return JSON containing the received student ID.

Conceptually:

```
{
  "student_id": 3,
  "message": "Student requested"
}
```

### Option B — Query parameter

Create:

```
GET /greet?name=Anita
```

Return something like:

```
{
  "message": "Hello Anita"
}
```

Do only one version for the exercise.

---

## 12. Concepts used

This small exercise uses:

```
Python function
FastAPI application
HTTP
GET
```

```
endpoint  
request  
response  
JSON  
path parameter  
or  
query parameter
```

The complete mental model is:

```
Client  
↓  
HTTP GET  
↓  
FastAPI endpoint  
↓  
Python function  
↓  
dictionary  
↓  
JSON response
```

---

## 13. Thought process

When creating a simple FastAPI endpoint, think in this order.

### Step 1: What should the client request?

Example:

```
GET /students/5
```

---

### Step 2: What information does the function need?

In this example:

```
student_id
```

---

### Step 3: Is it a path or query parameter?

If the URL looks like:

```
/students/5
```

use the path-parameter idea.

If it looks like:

```
/students?id=5
```

you are using a query parameter.

---

#### Step 4: What should the Python function return?

Keep it simple:

```
dictionary
```

For example:

```
student_id  
message
```

---

#### Step 5: What JSON should the client receive?

Think about the final response before writing the code.

Example:

```
{  
  "student_id": 5,  
  "message": "Student requested"  
}
```

---

## 14. Beginner-friendly pseudocode

For a path parameter:

```
START  
  
import FastAPI  
  
create FastAPI application  
  
create GET endpoint:  
    /students/{student_id}  
  
when client sends request:  
  
    receive student_id  
  
    create dictionary containing:  
        student_id  
        message
```

```
return dictionary
```

FastAPI converts result to JSON

```
END
```

For a query parameter:

```
START
```

```
create FastAPI application
```

```
create GET endpoint:  
    /greet
```

```
receive:  
    name query parameter
```

```
create message:  
    "Hello " + name
```

```
return dictionary containing message
```

```
END
```

---

## 15. Suggested solving approach: FastAPI

Keep the structure very small:

1. Import FastAPI
2. Create app
3. Create one GET endpoint
4. Define one Python function
5. Receive one parameter
6. Return one dictionary
7. Test the endpoint

You do not need:

```
database  
authentication  
multiple files
```

```
classes
Docker
deployment
```

today.

---

## 16. Easy edge case — missing query value

Suppose your endpoint expects:

```
/greet?name=...
```

but someone requests:

```
/greet
```

What should happen?

There are two beginner-friendly possibilities.

You might make the parameter required:

```
name must exist
```

or provide a default:

```
name = "Guest"
```

Then:

```
GET /greet
```

could conceptually return:

```
{
  "message": "Hello Guest"
}
```

The important lesson is:

Decide whether a query parameter is required or optional.

---

## 17. Easy edge case — invalid path value

Suppose your endpoint expects:

```
student_id
```

to be an integer.

Valid:

```
/students/5
```

Invalid:

```
/students/abc
```

If your function declares that `student_id` should be an integer, FastAPI can validate the incoming value.

Conceptually:

```
/students/5  
→ valid integer  
→ function runs
```

but:

```
/students/abc  
→ invalid integer  
→ validation error response
```

This is one helpful feature of FastAPI.

---

## 18. Expected request and JSON response

If you choose the path-parameter exercise:

### Request

```
GET /students/7
```

### Expected response

```
{  
  "student_id": 7,  
  "message": "Student requested"  
}
```

The flow is:

```
GET /students/7  
  ↓  
student_id = 7  
  ↓  
Python function
```

↓  
dictionary  
↓  
JSON response

Or for a query parameter:

## Request

GET /greet?name=Meera

## Expected response

```
{  
  "message": "Hello Meera"  
}
```

## 19. Hint only

Start with the simplest possible idea:

FastAPI app  
↓  
one GET endpoint  
↓  
one parameter  
↓  
one dictionary response

If using a path parameter, think:

/students/{student\_id}

If using a query parameter, think:

/greet?name=...

Then ask yourself:

What value arrives at my Python function?  
  
What dictionary should my function return?

For Day 118, remember this core flow:

Client Request  
↓  
FastAPI Endpoint

↓  
Python Function  
↓  
Dictionary  
↓  
JSON Response

Do not add authentication, databases, deployment, Docker, or asynchronous architecture yet.